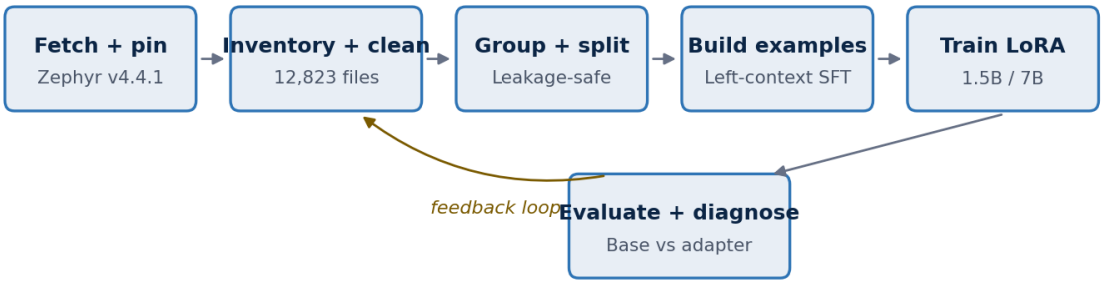
TECHNICAL PROJECT REPORT

Zephyr RTOS Code-Completion Fine-Tuning

Data engineering, LoRA adaptation, held-out evaluation,
C++ remediation, and five-GPU scaling



Prepared for technical assessment review

Status as of 16 August 2026 | Final held-out evaluation complete

Executive summary

Bottom line: The end-to-end assessment pipeline is operational and reproducible. Two adapters are prepared for publication. The original single-GPU 1.5B LoRA adapter provides the strongest broad Zephyr lift, reducing held-out general NLL by 19.17%, but it regresses on dedicated C++ likelihood. The validation-selected five-GPU 7B checkpoint improves held-out mean NLL by 7.12% on 512 general examples and 7.08% on 62 C++ examples, with positive token-accuracy and generation-similarity deltas on both benchmarks.

The project began as a technical assessment to adapt a small code model to idiomatic Zephyr RTOS completion. It evolved into a complete ML engineering workflow: source acquisition, repository inventory, license and duplicate analysis, leakage-safe splitting, left-context example construction, tokenizer-boundary validation, LoRA training, base-versus-adapter evaluation, checkpoint selection, failure analysis, targeted data reweighting, and distributed scaling.

Area	Current evidence	Assessment
Pipeline	All core data, training, and evaluation stages implemented and exercised	Complete
General Zephyr lift	NLL -19.17%; token accuracy +3.10 points; edit similarity +5.11 points	Proven on 1.5B
Dedicated C++ lift	Selected 7B adapter: NLL -7.08%; accuracy +0.46 points; first-line +4.84 points	Proven
C++ remediation	Effective C++ share increased from 4.57% to 19.92%; held-out sets unchanged	Implemented
7B scaling	Five-GPU DDP batch 2: 4.661 examples/s; checkpoint 100 selected after early stopping	Validated
Final 7B evaluation	General NLL -7.12%; C++ NLL -7.08%; positive generation deltas	Complete
Submission packaging	Separate 1.5B and 7B adapter archives, metadata, and checksums prepared; public hosting remains to verify	Prepared locally

Report map

- Sections 1-3 explain the assessment, RTOS/Zephyr context, and technical design.
- Sections 4-8 reconstruct the data, training, evaluation, and C++-remediation work chronologically.
- Sections 9-11 explain the 7B multi-GPU redesign, reproducibility controls, and current risks.
- The appendices provide metrics, key commands, artifact paths, and interview-ready talking points.

1. Assessment objective and acceptance criteria

The source assessment asks for a fine-tuning pipeline that adapts a small language model, such as Qwen 1.5B or DeepSeek-Coder 1.3B, to generate idiomatic Zephyr RTOS C++ under consumer-hardware constraints. The central grading criterion is demonstrated lift: the adapter must perform objectively better than the base model on held-out Zephyr code and provide at least one concrete side-by-side example.

Requirement	Implementation evidence	Status
Fetch official Zephyr source	Programmatic clone/fetch of tag v4.4.1, pinned to commit 1f6485e…	Met
Extract and clean .c/.cpp/.h/.hpp	13,697 files inventoried; 12,823 retained after explicit exclusion rules	Met
Train/held-out split	Parent-directory grouping plus hash isolation; leakage checks passed	Met
Code-completion format	Left-context prompt plus supervised continuation, maximum 1,024 tokens	Met
Hugging Face + LoRA	transformers, datasets, peft, trl, accelerate; rank-16 LoRA	Met
Base vs adapter evaluation	512 general and 62 C++ held-out examples; teacher-forced and generation metrics	Met
Quantitative lift	Selected 7B adapter improves held-out general and C++ NLL, accuracy, and generation similarity	Met
Side-by-side example	Multiple exact Zephyr completions where base hallucinated	Met, with caveat
Submission package	Public GitHub, environment file, README, and optional adapter link	To verify

Important scope note: The 7B experiment is an extension, not a substitute for the original 1.5B requirement. The 1.5B pipeline remains the assessment-compliant baseline; the 7B run tests whether model capacity can recover C++ quality after data remediation.

2. Technical foundations

2.1 RTOS and Zephyr RTOS

A real-time operating system (RTOS) is designed to provide predictable timing for embedded workloads. Unlike a desktop operating system optimized primarily for throughput and interactive convenience, an RTOS prioritizes bounded response times, deterministic scheduling, interrupt handling, synchronization, and operation within constrained memory and power budgets.

Zephyr is an open-source RTOS for microcontrollers and embedded devices. Its codebase combines architecture ports, kernel primitives, device drivers, devicetree-generated configuration, networking, Bluetooth, tests, samples, and platform integrations. Idiomatic Zephyr code therefore contains domain-specific macros and patterns such as DEVICE_DT_GET, DT_NODELABEL, Kconfig-guarded compilation, ztest assertions, driver callback tables, and devicetree-generated data structures.

- Determinism: operations are designed around predictable scheduling and interrupt latency.
- Resource constraints: code must fit small memory footprints and avoid unnecessary runtime overhead.
- Hardware abstraction: drivers and devicetree connect portable APIs to specific boards and SoCs.

- Configuration density: macros, compile-time feature flags, and generated bindings strongly influence valid code.

2.2 Left-context code completion

Each example is a pair: a prompt containing the code immediately before a cut point, and a completion containing the reference continuation. At inference time, the model receives only the prompt and predicts continuation tokens autoregressively. Training must therefore reproduce that boundary exactly.

Conceptual example

```
prompt = source_code[:cut]
completion = source_code[cut:reference_end]
```

The chosen context limit is 1,024 tokens. If prompt plus completion exceeds the limit, the pipeline removes the oldest prompt tokens while preserving the complete supervised continuation. This retains the code nearest the cut, which is usually most predictive, without truncating the target.

2.3 Supervised fine-tuning and completion-only loss

Supervised fine-tuning (SFT) teaches a causal language model to assign high probability to a known continuation. Prompt tokens provide context but are masked from the loss with label value -100. Completion tokens, including the end-of-sequence token, are supervised. This prevents the optimizer from spending capacity reproducing the supplied prompt.

Training representation

```
input_ids = prompt_ids + completion_ids + [eos_id]
labels    = [-100] * len(prompt_ids) + completion_ids + [eos_id]
```

2.4 LoRA: parameter-efficient adaptation

Low-Rank Adaptation (LoRA) freezes the original model weights and learns a low-rank update for selected linear layers. Instead of training a large weight matrix W directly, the update is represented as two smaller matrices: delta-W = B x A, scaled by alpha/r. Rank r controls adapter capacity; alpha controls the update scale.

Model	Total parameters	Trainable LoRA parameters	Trainable share
Qwen2.5-Coder-1.5B	1,562,179,072	18,464,768	1.1820%
Qwen2.5-Coder-7B	7,655,986,688	40,370,176	0.5273%

Both configurations used rank 16, alpha 32, dropout 0.05, no bias adaptation, and all eligible linear modules. This includes attention projections and feed-forward projections while preserving the frozen base model.

2.5 Evaluation metrics

Metric	Meaning	Direction
Mean negative log-likelihood (NLL)	Average surprise assigned to the reference completion	Lower
Perplexity	exp(NLL); effective branching uncertainty	Lower
Completion-token accuracy	Share of reference tokens predicted as the top next token	Higher

Metric	Meaning	Direction
Exact match	Generated token sequence equals the reference	Higher
First-line exact match	First generated line equals the reference first line	Higher
Token edit similarity	Normalized similarity between generated and reference tokens	Higher
Matching-prefix ratio	Fraction of the reference matched from its beginning	Higher

Teacher-forced metrics answer whether the model assigns better probability to the correct continuation. Greedy-generation metrics answer whether that improved distribution produces better actual output. Both matter: a model can lower NLL without improving exact generation, or match a first line while degrading later tokens.

3. Reproducible environment and source pinning

Component	Recorded value
Host OS	Linux 5.15.0-112-generic, x86_64, glibc 2.35
Python	3.11.10
CPU / RAM	80 visible CPU cores; 93.0 GB RAM
GPU	5 x NVIDIA TITAN RTX, 24,576 MiB each, compute capability 7.5
PyTorch	2.9.1+cu128; CUDA available
Transformers	5.14.1
Datasets / PEFT / TRL	5.0.0 / 0.19.1 / 1.8.0
Accelerate	1.14.0
Project root	/home/ahmadalmoustafa/zephyr-code-finetune
Seed	20260717

Pinned inputs

- Zephyr source: tag v4.4.1, resolved commit 1f6485eca25431b5ff27ce9a754218c9e559bbbb.
- Qwen2.5-Coder-1.5B revision: df3ce67c0e24480f20468b6ef2894622d69eb73b.
- Qwen2.5-Coder-7B revision: 0396a76181e127dfc13e5c5ec48a8cee09938b02.

Pinning prevents silent drift. A model name or repository tag can later point to different bytes; recording resolved commits makes future reproduction and metric comparisons meaningful.

4. Data collection, inventory, and cleaning

4.1 Repository acquisition

The official Zephyr repository was fetched programmatically into data/raw/zephyr. A repository manifest records the requested tag, resolved commit, source URL, and destination. This provides a traceable boundary between upstream source and derived datasets.

4.2 Source inventory

Extension	Files	Lines	Size
.c	8,553	3,026,275	79.74 MB
.h	5,069	872,427	29.65 MB
.cpp	59	8,403	0.30 MB
.hpp	16	981	0.03 MB
Total	13,697	3,908,086	109.7 MB

Explicit C++ represented only 75 source files before cleaning, approximately 0.548% of the source inventory. This imbalance later became the main modeling risk: a generally improved Zephyr model could still fail the specific C++ objective.

4.3 Quality and licensing analysis

- 96 exact-duplicate hash groups were detected, containing 145 extra duplicate files.
- 13,591 files declared Apache-2.0; smaller MIT and BSD groups were retained; 46 files had missing SPDX identifiers and require submission-time review.
- Generated macro headers, oversized files, empty files, vendored paths, test fixtures, and hex-heavy model-data sources were identified explicitly rather than removed by an opaque heuristic.
- C++ files were classified into direct Zephyr, Zephyr-pattern, generic/third-party, and generated/data-heavy groups, then assigned high, medium, or low training priority.

4.4 Cleaning outcome

Disposition	Files	Reason / scope
Included	12,823	99.2 MB; 3,684,975 lines
Excluded	874	679 path-prefix, 135 duplicate, 54 hex-heavy, 3 generated/vendor, 2 oversized, 1 empty
Included C/C++	8,396 .c; 4,358 .h	Core domain coverage
Included explicit C++	53 .cpp; 16 .hpp	69 files; 43 high, 6 medium, 20 low priority

Why exact duplicate removal matters: If identical source appears in both training and evaluation, the model can score well by memorization rather than learning Zephyr conventions. Hash-level deduplication and group-aware splitting make the measured lift credible.

5. Leakage-safe splitting and example construction

5.1 Parent-directory grouping

The 12,823 cleaned files were organized into 3,501 parent-directory groups. Entire groups were assigned to train, validation, or evaluation so neighboring files from the same component could not cross the split. This is stricter than random file splitting and better reflects transfer to unseen components.

Split	Files	Groups	Data share	Explicit C++
Train	10,253	2,806	80.00%	41
Validation	1,331	349	10.00%	12
Evaluation	1,239	346	10.00%	16

- No duplicate path crossed splits.
- No parent-directory group crossed splits.
- No exact source hash crossed splits.

5.2 Completion dataset

Split	Examples	Files represented	Avg prompt tokens	Avg completion tokens	C++ examples
Train	64,935	10,015	525.69	106.16	183
Validation	4,774	1,298	428.03	99.22	47
Evaluation	4,517	1,211	452.69	101.16	62

Examples were deduplicated by prompt/completion fingerprint. Files too short or unsuitable for a valid cut were recorded rather than silently dropped. The initial training-repeat policy produced 67,850 effective training examples and an effective C++ sampling share of 4.57%.

5.3 Structural validation

- Unique example IDs: pass.
- Unique prompt/completion fingerprints: pass.
- Token limits: pass.
- Directory-group and source-hash isolation: pass.
- Independent token-count spot check: 500 examples, zero mismatches.

5.4 Held-out benchmarks

Benchmark	Examples	Source files	Composition	Purpose
General	512	512	322 C, 174 headers, 10 C++, 6 C++ headers	Broad Zephyr lift
C++	62	16	38 .cpp, 24 .hpp; 19 high-priority	Dedicated C++ objective

The C++ benchmark intentionally contains every held-out C++ completion example, while the general benchmark samples one example per source file for breadth. Checkpoint sweeps used validation data only and explicitly did not read the held-out evaluation benchmarks.

6. Tokenization boundary bug and correction

Observed defect: When prompt and completion text were tokenized together, 17.16% of 5,000 training examples and 17.70% of 1,000 validation examples changed exactly one prompt-boundary token.

Qwen2 uses a byte-level BPE tokenizer. BPE can merge characters across the prompt/completion boundary, especially around newline and indentation. During real generation, the prompt is already tokenized before the model emits its first completion token, so a cross-boundary merge cannot occur. Training on the combined text therefore created a subtle mismatch between training and inference.

Boundary-safe construction

```
# Incorrect for boundary-faithful completion training
combined_ids = tokenizer(prompt + completion)

# Correct
prompt_ids = tokenizer(prompt, add_special_tokens=False)
completion_ids = tokenizer(completion, add_special_tokens=False)
input_ids = prompt_ids + completion_ids + [eos_id]
```

The training script was updated to tokenize prompt and completion separately, concatenate token IDs, mask every prompt label with -100, supervise EOS, and validate exact decode round trips. The corrected smoke test passed all 256 examples, and pretokenized token counts matched independent checks.

- This was not a cosmetic difference: the changed token was the first token the model was asked to predict.
- The fix aligns data construction, autoregressive generation, and evaluation.
- The pipeline now fails early if a completion occupies the entire context or has no supervised tokens.

7. Initial 1.5B LoRA training

7.1 Trainer compatibility and smoke tests

The installed TRL/Transformers versions differed from older examples: SFTConfig no longer accepted overwrite_output_dir. The script now introspects the installed SFTConfig signature, keeps required fields strict, and omits unsupported optional fields with a logged warning.

Smoke configuration	Runtime	Peak allocated	Train loss	Validation loss	Outcome
Batch 2, grad accumulation 4	39.73 s	4.77 GB	0.9391	1.0761	Pass
Batch 8, grad accumulation 1	44.99 s	5.60 GB	0.9392	1.0759	Pass

The nearly identical losses confirmed that changing microbatch geometry did not alter the objective. Batch 2 was slightly faster on this hardware despite lower instantaneous VRAM use, showing why measured throughput is more reliable than assuming a larger batch is faster.

7.2 Full adapter integrity

The full 1.5B run produced a valid PEFT adapter at outputs/zephyr_lora/final_adapter. Integrity checks confirmed adapter_config.json and adapter_model.safetensors, an 82 MB artifact directory, rank 16, alpha 32, inference mode enabled, and the expected target modules: q/k/v/o projections plus gate/up/down projections.

8. Evaluation results and diagnosis

8.1 Full held-out evaluation of the initial 1.5B adapter

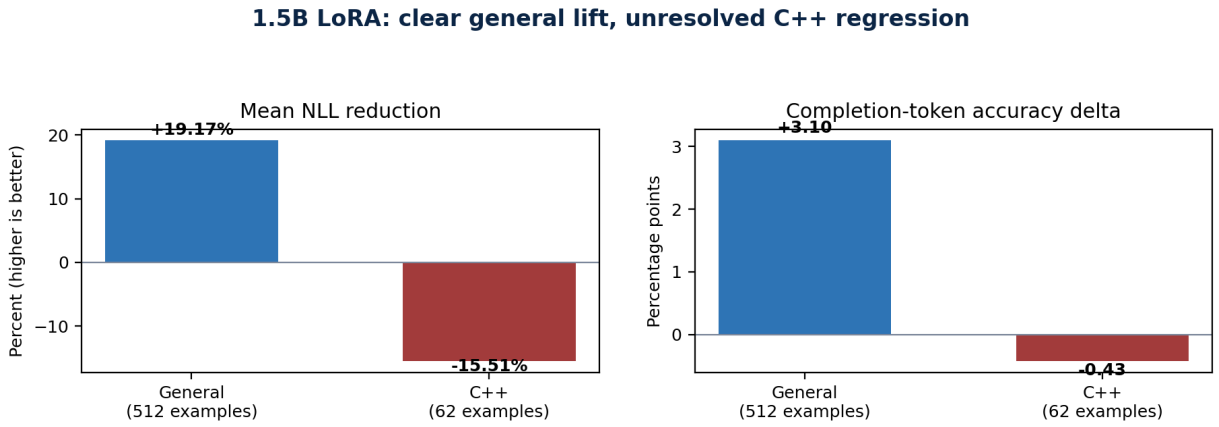


Figure 1. Full held-out evaluation: the adapter improved broad Zephyr completion but regressed on the dedicated C++ subset.

Benchmark / metric	Base	1.5B LoRA	Lift
General mean NLL	0.950232	0.768027	+19.17% reduction
General perplexity	2.5863	2.1555	+16.66% reduction
General token accuracy	78.88%	81.98%	+3.10 points
General exact match	0.39%	4.49%	+4.10 points
General first-line match	35.55%	40.82%	+5.27 points
General edit similarity	28.62%	33.74%	+5.11 points
C++ mean NLL	0.917017	1.059281	-15.51% reduction
C++ perplexity	2.5018	2.8843	-15.29% reduction
C++ token accuracy	80.09%	79.67%	-0.43 points
C++ first-line match	51.61%	53.23%	+1.61 points
C++ edit similarity	33.95%	33.40%	-0.55 points

Interpretation: the adapter clearly learned broad Zephyr syntax and recurring platform patterns, but the training distribution was dominated by C and headers. The dedicated C++ benchmark exposed that the model had not learned a sufficiently robust C++ specialization. Because demonstrated C++ lift is the crucial assessment criterion, the correct conclusion was not to select this adapter as final.

8.2 Concrete side-by-side success

On a held-out Bluetooth test server, the base model invented another test table while the adapter produced the exact Zephyr entry point. This is a valid Zephyr-domain win and a strong side-by-side example, although it comes from the general benchmark rather than an explicit .cpp file.

Base continuation (incorrect)

```
static const struct bst_test_instance test_def2[] = {
    {
```

LoRA continuation (exact reference)

```
int main(void)
{
    bst_main();

    return 0;
}
```

A second exact win occurred in soc/renesas/rz/rzt2l/soc.h: the base emitted nonsensical self-referential IRQn macros, while the LoRA model correctly emitted the CMSIS feature macros, the Zephyr-owned IRQ declaration, and the closing include guard.

Caveat: These examples satisfy the required side-by-side Zephyr comparison, although they come from the initial 1.5B general benchmark rather than an explicit .cpp file. The selected 7B experiment separately demonstrates aggregate held-out C++ lift; source-file-clustered uncertainty and compiler-based evaluation remain future improvements.

8.3 Checkpoint sweeps

A validation-only sweep compared saved checkpoints without reading the held-out evaluation benchmarks. For the original training run, checkpoint-8000, checkpoint-8482, and final_adapter all worsened high-priority C++ NLL relative to the base model. The selection policy therefore returned no recommended checkpoint and requested sampling or training changes.

After C++ reweighting, a larger five-GPU sweep evaluated checkpoints 200 through 2527. Some checkpoints improved validation token accuracy by roughly 0.8-0.9 points, but high-priority C++ NLL still worsened. Checkpoint 200 was chosen only as the least risky candidate for final held-out diagnosis, not as a validated winner.

8.4 Held-out evaluation of C++20 checkpoint 200

Benchmark	NLL reduction	Accuracy delta	Edit-similarity delta	Conclusion
General, 512	+7.58%	+0.87 points	+1.36 points	Modest lift
C++, 62	+1.52%	-0.09 points	+0.00 points	Likelihood improved; generation flat

Checkpoint 200 reduced C++ NLL from 0.917017 to 0.903065, but token accuracy moved from 80.09% to 80.00%, and greedy edit similarity remained 33.95%. This is evidence of a better probability distribution, but not enough to claim practically better C++ completion.

9. C++ data remediation

9.1 Root cause

Only 69 explicit C++ files survived cleaning, and the original training examples contained 183 explicit C++ completions among 64,935 raw training examples. Even with priority repeats, the effective C++ sampling share was only 4.57%. A model could minimize overall loss by becoming better at C and headers while barely updating its C++ behavior.

9.2 Priority-aware reweighting

Group	Raw examples	Effective examples	Policy
Non-C++	64,752	64,752	1x
C++ medium	21	336	16x
C++ high	140	15,680	112x
C++ low	22	88	4x
Total	64,935	80,856	19.92% effective C++

The new train_cpp20.jsonl changes only training repeats. Validation and evaluation remain byte-for-byte untouched. This is essential: changing the held-out sets after seeing poor results would invalidate the comparison.

Trade-off: Oversampling increases the probability that the optimizer sees scarce C++ examples, but it does not create new source diversity. Excessive repetition can overfit exact files, so checkpoint sweeps and untouched evaluation data remain necessary.

9.3 Full C++20 1.5B retraining

Metric	Value
Effective training examples	80,856
Optimizer steps	2,527
Training runtime	32,230 s / 539.05 min / 8 h 59 min
Training loss	0.65055
Validation loss	0.85571
Peak allocated VRAM	14.73 GB on the lead shard
Execution	Single Python process, model sharded across five GPUs with device_map='auto'

Although all five GPUs held model shards, utilization was unbalanced because a single batch traversed layer partitions sequentially. nvidia-smi showed one Python PID on every GPU, a signature of model parallelism rather than data-parallel throughput.

10. Moving to Qwen2.5-Coder-7B

10.1 Why a larger model was tested

The original task does not require a 7B model, and the 1.5B pipeline should remain the primary assessment deliverable. The 7B experiment is a controlled extension motivated by capacity: a larger pretrained code model may better preserve C++ syntax and abstractions while adapting to Zephyr-specific conventions.

- Potential benefit: stronger pretrained representation of templates, namespaces, RAII, overloads, and mixed C/C++ build patterns.
- Cost: approximately five times more base parameters, higher VRAM demand, and longer training/evaluation.
- Scientific requirement: evaluate on exactly the same held-out benchmarks; do not infer improvement from training loss alone.

10.2 Why device_map='auto' was slow

Automatic device placement solved memory capacity by putting different model layers on different GPUs. It did not make five independent batches run in parallel. Each forward and backward pass still moved through the layer sequence, producing low or uneven utilization and an estimated 16.9-hour full run at roughly 1.33 examples per second.

This distinction is fundamental: model parallelism answers how one model can fit, while data parallelism answers how several batches can be processed simultaneously.

10.3 Five-GPU DDP through Accelerate

The training script gained a second execution strategy using DistributedDataParallel (DDP) launched by accelerate launch. Each of five processes owns one complete 7B model replica on one TITAN RTX. Each process receives a different slice of the dataset, computes LoRA gradients locally, and synchronizes those gradients at optimizer boundaries.

- No torchrun command is required; Accelerate provides the launcher and process environment.
- Model loads are serialized across ranks to avoid five simultaneous 7B CPU copies exhausting 93 GB of system RAM.
- Gradient checkpointing trades extra recomputation for lower activation memory.
- Chunked NLL reduces peak language-model-head memory and is compatible with ordinary DDP-bound forward methods.
- Legacy single-process DataParallel is explicitly disabled for the auto-sharded path.

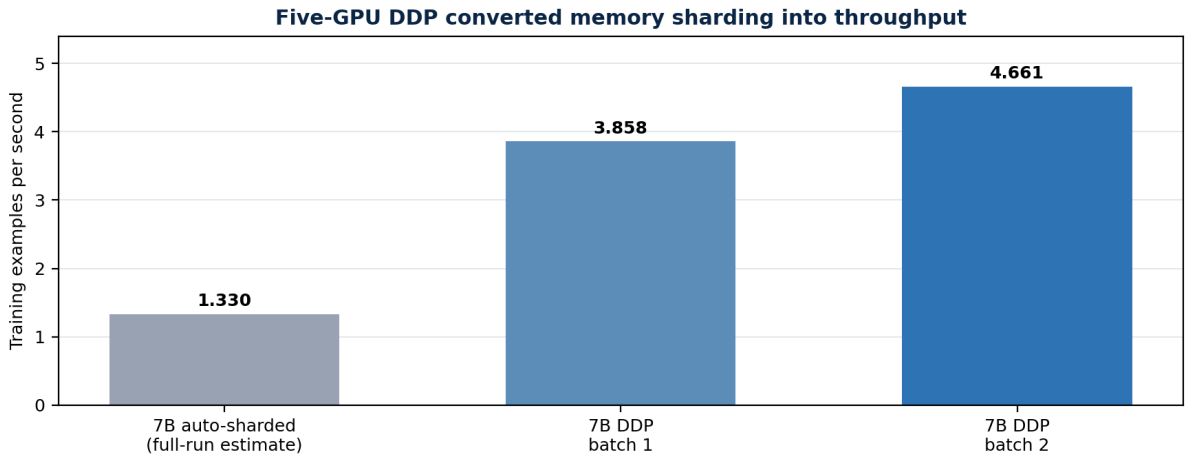


Figure 2. Measured throughput improvement from DDP. The auto-sharded value is derived from the observed 24.07-second step at effective batch 32.

10.4 DDP smoke-test selection

7B configuration	Examples processed	Throughput	Peak reserved	Train loss	Validation loss
DDP batch 1, grad 1	100	3.858 ex/s	18.69 GB	0.86761	1.06810
DDP batch 2, grad 1	200	4.661 ex/s	18.68 GB	0.81909	1.05087

Batch 2 increased measured throughput by 20.8% over batch 1 while peak reserved memory stayed at 18.68 GB. It was therefore selected for the full run. The full geometry is batch 2 per GPU x gradient accumulation 3 x 5 GPUs = effective global batch 30, with 2,696 optimizer steps and 81 warmup steps.

Projected runtime: At the measured batch-2 rate, processing 80,856 effective examples should take approximately 4.8 hours before checkpoint and final-evaluation overhead. A practical estimate is 4.7-5.0 hours, versus about 16.9 hours for the auto-sharded approach.

The NCCL warning about guessed device IDs did not prevent either smoke run on this homogeneous single-node mapping. It can be removed later by passing the local CUDA device explicitly when initializing the process group. Repeated tokenizer and token-ID alignment messages are expected once per rank; the unauthenticated Hub warning affects download rate limits, not training correctness.

10.5 Early stopping and final 7B held-out evaluation

The nominal one-epoch schedule contained 2,696 optimizer steps at effective global batch 30. Because the reweighted training set repeats 140 high-priority C++ examples 112 times each, a full effective epoch carried substantial overfitting risk. Training was therefore stopped after checkpoint 1000 was safely written, and checkpoints 100 through 1000 were compared using only the 47-example C++ validation set.

Checkpoint 100 passed the all-C++ accuracy guardrail and reduced high-priority validation NLL from 0.682454 to 0.641194, a 6.05% improvement. Later checkpoints worsened NLL. At 3,000 effective example presentations, the selected checkpoint used only 3.71% of one effective epoch.

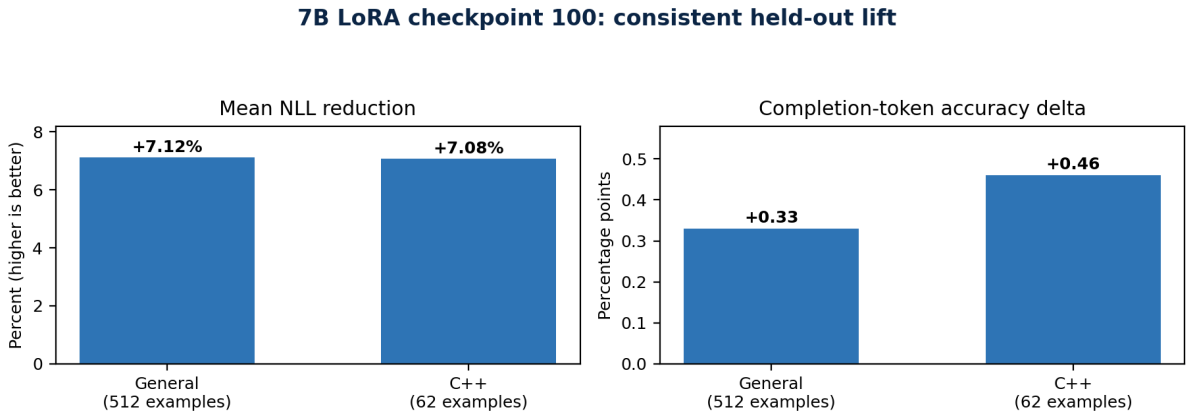


Figure 3. The validation-selected 7B adapter improved both held-out benchmarks, resolving the 1.5B C++ regression.

Benchmark / metric	7B base	7B LoRA	Lift
General mean NLL	0.740071	0.687411	+7.12% reduction
General token accuracy	82.79%	83.12%	+0.33 points
General exact match	0.59%	3.12%	+2.53 points
General first-line match	40.23%	42.97%	+2.74 points
General edit similarity	32.91%	35.00%	+2.08 points
C++ mean NLL	0.653255	0.606977	+7.08% reduction
C++ token accuracy	84.55%	85.01%	+0.46 points
C++ first-line match	59.68%	64.52%	+4.84 points
C++ edit similarity	36.50%	37.69%	+1.19 points

Final result: The selected Qwen2.5-Coder-7B LoRA adapter produced consistent lift on general and dedicated C++ held-out data. This is the strongest final model. The 1.5B workflow remains the assessment-compliant small-model baseline and documents the complete consumer-hardware LoRA pipeline.

11. Reproducibility and MLOps controls

Control	Implementation	Why it matters
Version pinning	Repository and model commit hashes	Prevents silent upstream drift
Deterministic split	Seed 20260717 plus directory grouping	Repeatable train/validation/evaluation boundaries
Manifest chain	Repository, inventory, cleaning, split, dataset, and benchmark summaries	Auditable transformation history
Leakage checks	Path, directory-group, source-hash, and fingerprint isolation	Protects evaluation credibility
Smoke mode	Small dataset, 20 steps, full validation path	Catches API, memory, and masking failures cheaply
Pinned base evaluation	Same base revision and tokenizer for base and LoRA	Fair lift measurement
Validation-only sweep	Evaluation benchmark explicitly not read	Avoids test-set checkpoint selection
Persistent logs	nohup, unbuffered output, report logs, PID files	Survives disconnects and supports diagnosis
Run summaries	JSON with config, metrics, versions, memory, and paths	Machine-readable provenance
Adapter integrity	Required files and PEFT configuration verified	Portable inference artifact

12. Current limitations and remaining work

1. Add source-file-clustered bootstrap confidence intervals. The C++ benchmark has 62 examples but only 16 source files, so an example-level confidence interval would overstate independence.
2. Add compiler- or test-based functional evaluation. Text similarity cannot prove that generated embedded code compiles, links, or behaves correctly.
3. Review the 46 files without SPDX identifiers and document dataset redistribution policy before publishing derived data or adapter weights.
4. Finalize the required public GitHub repository, environment specification, README reproduction guide, measured-lift section, and optional adapter download link.

Remaining scientific risk: The positive C++ result is based on 62 completions from 16 source files. It is valid held-out evidence, but uncertainty should be clustered by source file and supplemented with compilation or Zephyr test execution before claiming production readiness.

13. Final model decision

Model	Observed result	Submission role
Qwen2.5-Coder-1.5B final adapter	Single-GPU run; general NLL -19.17% and accuracy +3.10 points; dedicated C++ NLL regressed	Published small-model adapter with strongest broad Zephyr lift
Qwen2.5-Coder-7B checkpoint 100	Positive held-out NLL, accuracy, first-line, and edit-similarity lift on general and C++	Published best-performing extension and C++-capable adapter

Submission claim: Parameter-efficient fine-tuning demonstrably improved pinned code models on held-out Zephyr RTOS completion. The published original 1.5B adapter proves the requested small-model, single-GPU pipeline and delivers the strongest broad-domain lift. The separately published validation-selected 7B adapter resolves the dedicated C++ weakness and is the strongest balanced final model.

Appendix A. Chronological project record

Step	Work completed
1	Initialized writable project structure and confirmed Linux, Python, Git, disk, and GitHub connectivity.
2	Fetched Zephyr v4.4.1 and recorded commit 1f6485e... in repository.json.
3	Inventoried 13,697 C/C++ files, extensions, directories, licenses, sizes, and exact duplicates.
4	Classified explicit C++ files by Zephyr relevance and training priority.
5	Cleaned to 12,823 files using explicit path, duplicate, generated-data, size, and empty-file rules.
6	Created parent-directory train/validation/evaluation splits and passed leakage checks.
7	Pinned and validated the Qwen2.5-Coder-1.5B tokenizer.
8	Constructed 64,935 train, 4,774 validation, and 4,517 evaluation completion examples.
9	Validated example identities, token limits, split isolation, and token counts.
10	Created 512-example general and 62-example C++ held-out benchmarks.
11	Audited hardware and Hugging Face training APIs on five TITAN RTX GPUs.
12	Detected a 17% BPE boundary mismatch and corrected training to tokenize prompt/completion separately.

Step	Work completed
13	Validated the 1.5B LoRA trainer with smoke tests and corrected version-specific SFTConfig behavior.
14	Completed 1.5B adapter training, verified artifact integrity, and ran full base-versus-LoRA evaluation.
15	Observed strong general lift but dedicated C++ regression; refused to claim complete success.
16	Swept checkpoints on validation only; no original checkpoint satisfied the C++ selection criteria.
17	Built train_cpp20.jsonl with 19.92% effective C++ share while preserving validation/evaluation.
18	Completed C++20 retraining and a 26-candidate validation checkpoint sweep.
19	Evaluated checkpoint 200 on held-out data: modest general lift, slight C++ NLL gain, flat generation.
20	Pinned Qwen2.5-Coder-7B and validated LoRA smoke training.
21	Diagnosed device_map='auto' as memory sharding rather than throughput scaling.
22	Added Accelerate DDP with one complete model replica per GPU and serialized model loading.
23	Validated DDP batch 1, then batch 2; selected batch 2 for 20.8% higher throughput.
24	Prepared the full 7B DDP configuration: effective batch 30, 2,696 steps, projected 4.7-5.0 hours.
25	Launched the five-GPU 7B run and retained checkpoints 100 through 1000.
26	Stopped early after recognizing that one reweighted epoch would repeat high-priority C++ examples excessively.
27	Swept ten 7B checkpoints using only 47 C++ validation examples across five GPU workers.
28	Selected checkpoint 100: high-priority validation NLL improved 6.05% while the all-C++ accuracy guardrail passed.
29	Ran one final untouched evaluation: general NLL improved 7.12% and C++ NLL improved 7.08%, with positive generation deltas.

Appendix B. Key commands for the selected 7B run

Full five-GPU DDP launch with persistent logging

```
PROJECT=/home/ahmadalmoustafa/zephyr-code-finetune
LOG="$PROJECT/reports/training_7b_fast_ddp_batch2_full.log"

nohup env \
 CUDA_VISIBLE_DEVICES=0,1,2,3,4 \
 PYTHONUNBUFFERED=1 \
 OMP_NUM_THREADS=4 \
 accelerate launch \
  --multi_gpu \
  --num_processes 5 \
  --num_machines 1 \
  --gpu_ids 0,1,2,3,4 \
  --mixed_precision fp16 \
  --main_process_port 29504 \
  "$PROJECT/scripts/train_lora.py" \
  --mode full \
  --model-config "$PROJECT/configs/model_7b.json" \
  --train-config "$PROJECT/configs/train_lora_7b_fast_ddp_batch2.json" \
> "$LOG" 2>&1 < /dev/null &

echo $! | tee "$PROJECT/reports/training_7b_fast_ddp_batch2_full.pid"
tail -f "$LOG"
```

Validation-only selection across ten checkpoints

```
CUDA_VISIBLE_DEVICES=0,1,2,3,4 python -u \
 "$PROJECT/scripts/sweep_cpp_checkpoints.py" \
 --mode full \
 --model-config "$PROJECT/configs/model_7b.json" \
 --validation-data "$PROJECT/data/processed/validation.jsonl" \
 --training-output "$PROJECT/outputs/zephyr_lora_7b_fast_ddp_batch2" \
 --output-dir "$PROJECT/results/cpp_checkpoint_sweep_7b_batch2_full" \
 --gpus 0,1,2,3,4 --batch-size 2
```

One final untouched held-out evaluation

```
CUDA_VISIBLE_DEVICES=0 python -u \
 "$PROJECT/scripts/evaluate_models.py" \
 --mode full \
 --model-config "$PROJECT/configs/model_7b.json" \
 --adapter "$PROJECT/outputs/zephyr_lora_7b_fast_ddp_batch2/checkpoint-100" \
 --output-dir "$PROJECT/results/evaluation_7b_checkpoint100" \
 --loss-batch-size 2 --generation-batch-size 4 --max-new-tokens 128
```

Appendix C. Principal artifacts

Category	Representative path	Purpose
Raw source	data/raw/zephyr	Pinned Zephyr checkout
Repository manifest	data/manifests/repository.json	Source URL, tag, commit
Inventory	data/manifests/source_files.jsonl	Per-file metadata and hashes
Clean manifest	data/manifests/cleaned_files.jsonl	Inclusion/exclusion decisions
Split manifest	data/manifests/split_files.jsonl	Leakage-safe assignments
Training data	data/processed/train.jsonl	Original left-context examples
C++20 training data	data/processed/train_cpp20.jsonl	Priority-weighted repeats
Benchmarks	data/processed/benchmark_general.jsonl; benchmark_cpp.jsonl	Held-out evaluation
Trainer	scripts/train_lora.py	Auto-sharded and DDP LoRA training
Evaluator	scripts/evaluate_models.py	Base-versus-adapter metrics and examples
Checkpoint sweep	scripts/sweep_cpp_checkpoints.py	Validation-only model selection
Reweighting	scripts/prepare_cpp_retraining_data.py	C++ priority sampling
Published 1.5B adapter	outputs/zephyr_lora/final_adapter	Original single-GPU adapter; strongest general Zephyr lift
7B DDP config	configs/train_lora_7b_fast_ddp_batch2.json	Selected full-run geometry
Published 7B adapter	outputs/zephyr_lora_7b_fast_ddp_batch2/checkpoint-100	Validation-selected balanced final adapter
7B checkpoint sweep	results/cpp_checkpoint_sweep_7b_batch2_full	Validation-only selection evidence
Final 7B evaluation	results/evaluation_7b_checkpoint100	Untouched benchmark metrics and predictions
Reports and predictions	results/* and reports/*	Metrics, examples, logs, provenance

Appendix D. Interview-ready technical talking points

- Why grouping beats random splitting: adjacent Zephyr files share macros and structure; directory grouping reduces near-neighbor leakage.
- Why tokenization was separated: BPE merges can cross a text boundary during offline concatenation but cannot cross the generation boundary at inference.
- Why NLL and generation were both reported: probability calibration and greedy output can move differently.
- Why the C++ regression is valuable evidence: the benchmark found a real distributional weakness and prevented a misleading success claim.
- Why oversampling was priority-aware: scarce direct-Zephyr C++ is more relevant than third-party or data-heavy C++.
- Why device_map auto was not enough: it solved model capacity, not parallel example throughput.

- Why DDP is recruiter-relevant: five independent processes, one model per GPU, distributed sampling, gradient synchronization, and measured throughput scaling.
- Why checkpoint 100 won: it minimized high-priority validation NLL, passed the all-C++ guardrail, and later checkpoints showed overfitting.
- Why the final claim is credible: checkpoint selection used validation only, and the 512-example general plus 62-example C++ evaluation benchmarks were read once afterward.

Appendix E. Evidence basis and references

This report is reconstructed from the assessment PDF, repository and data-processing outputs, training logs, generated evaluation reports, validation-only checkpoint sweeps, adapter integrity checks, five-GPU DDP evidence, and the final 7B held-out evaluation provided through 16 August 2026.

Evidence	Recorded source
Assessment	Technical Assessment: Embedded OS Code Completion Fine-Tuning (2-page PDF)
Source	Official Zephyr repository, tag v4.4.1, commit 1f6485e…
Model	Qwen/Qwen2.5-Coder-1.5B and Qwen/Qwen2.5-Coder-7B, pinned revisions
Primary evaluation	results/evaluation_7b_checkpoint100/evaluation_report.md and evaluation_summary.json
Training evidence	training.log, run summaries, adapter integrity output
C++ remediation	prepare_cpp_retraining_data.py output and cpp20 checkpoint sweep
DDP evidence	batch-1 and batch-2 smoke logs, full-run checkpoints, nvidia-smi process/memory snapshots
7B model selection	cpp_checkpoint_sweep_7b_batch2_full report and per-example metrics

External project links: Zephyr | Qwen2.5-Coder-1.5B | Qwen2.5-Coder-7B